Lab 7: Fuzzing for Input Validation Bugs

Lab report on *mobile_data_plan.py* using Hypothesis and pytest

Name	Akshay Kumar M
SRN	PES1UG24CS045
Date	5 November 2026

1. Aim

The aim of this lab was to find input validation bugs in a small Python program by fuzzing it with Hypothesis, fix the program, and then confirm with the same tests that the bugs were gone. My program was *mobile_data_plan.py*. It has three functions: `parse_plan`, `usage_average` and `payable_bill`. Each one takes a single string and works on it.

2. Setup

I worked in a Python virtual environment on a MacBook (Python 3.14.3) and installed the two packages the handout asks for:

```
pip install hypothesis
pip install pytest
```

The versions pytest reported were pytest 9.2.0 and Hypothesis 6.170.14.

3. Running the original program (Step 2)

I first ran the script as it was given to me, with normal inputs, to see what it does when nothing goes wrong. The three functions ask for input one after another.

```
(.venv) akshaykumarm@AKSHAYs-MacBook-Air-2 Lab 7 % python mobile_data_plan.py
==== Running parse_plan ====
Enter plan:data in GB (e.g. standard:5): standard:5
Plan and Data: ('Standard', 5.0)

==== Running usage_average ====
Enter daily usage in MB as CSV (e.g. 300,450,800): 300,450,800
Average Usage: 516

==== Running payable_bill ====
Enter bill|discount (e.g. 399|50): 399|50
Payable Bill: 420.82
(.venv) akshaykumarm@AKSHAYs-MacBook-Air-2 Lab 7 %
```

Figure 1. Output of the original program with normal input.

What each function did with the inputs I typed:

Function	Input	Output
parse_plan	standard:5	('Standard', 5.0)
usage_average	300,450,800	516
payable_bill	399|50	420.82

Two of these results are worth a note. The mean of 300, 450 and 800 is 516.67, but the program prints 516, so the average is cut down to a whole number. For the bill, 399 × 1.18 = 470.82 and 470.82 – 50 = 420.82, so it looks like the function adds 18% tax and then takes off the discount as a flat amount. I worked that out from the output; I did not change either behaviour, since the handout only asks for crashes to be fixed.

So with well-formed input the program works. That is exactly the situation where unit tests usually stop, and where fuzzing starts.

4. Writing the fuzz tests (Step 3)

In *test_mobile_data_plan.py* I wrote one Hypothesis test for each function. They all check the same property: whatever string comes in, the function must not raise an exception. I did not check the returned values here, only that nothing crashes. Following the handout, the inputs were drawn from `st.text() | st.none()`, so Hypothesis tries random Unicode text and also `None`. The tests are named `test_parse_plan_no_crash`, `test_usage_average_no_crash` and `test_payable_bill_no_crash`.

5. Running the tests on the buggy code (Step 4)

I ran `python -m pytest -v test_mobile_data_plan.py`. All three tests failed.

```
(.venv) akshaykumarm@AKSHAYs-MacBook-Air-2 Lab 7 % python -m pytest -v test_mobile_data_plan.py
E           Failed: Unexpected exception: AttributeError: 'NoneType' object has no attribute 'split'
E           Failing test case: test_payable_bill_no_crash(
E               s=None,
E           )
E           Explanation:
E               These lines were always and only run by failing test cases:
E                   /Users/akshaykumarm/Downloads/L6_L7_45_mobile_data_plan/Lab 7/test_mobile_data_plan.py:12

test_mobile_data_plan.py:13: Failed
=========================================== short test summary info ============================================
FAILED test_mobile_data_plan.py::test_parse_plan_no_crash - Failed: Unexpected exception: AttributeError: 'NoneType' object has
 no attribute 'split'
FAILED test_mobile_data_plan.py::test_usage_average_no_crash - Failed: Unexpected exception: AttributeError: 'NoneType' object
has no attribute 'split'
FAILED test_mobile_data_plan.py::test_payable_bill_no_crash - Failed: Unexpected exception: AttributeError: 'NoneType' object h
as no attribute 'split'
============================================ 3 failed in 0.21s =============================================
(.venv) akshaykumarm@AKSHAYs-MacBook-Air-2 Lab 7 %
```

Figure 2. All three fuzz tests failing against the original code (3 failed in 0.21s).

The error was the same in every case: `AttributeError: 'NoneType' object has no attribute 'split'`. Hypothesis reduced the failing input to the smallest one it could, and it printed `s=None`. That means each function calls `.split()` on its argument straight away, without first checking that it really got a string. Normal typing at the prompt can never produce `None`, which is why my manual run in Figure 1 looked fine and the bug stayed hidden.

This is the same pattern as the `parse_int_list(None)` example in the handout. One mistake was repeated in three places.

6. Fixing the program (Step 5)

The fix was to stop trusting the input. Each function now checks the argument before it splits it, and handles a missing or unusable value without raising an exception. After the change I ran the script again with the same three inputs as before.

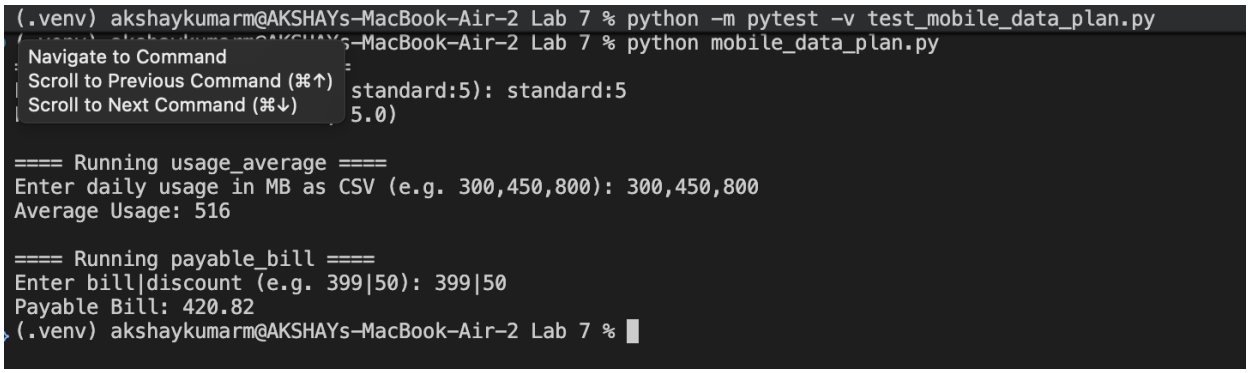


Figure 3. Output of the fixed program. The results match Figure 1.

The outputs are identical to the original run (`('Standard', 5.0)`, 516 and 420.82), so the hardening did not change behaviour for valid input.

7. Re-running the tests on the fixed code (Step 6)

I then ran the same test file again, without changing a line of it.

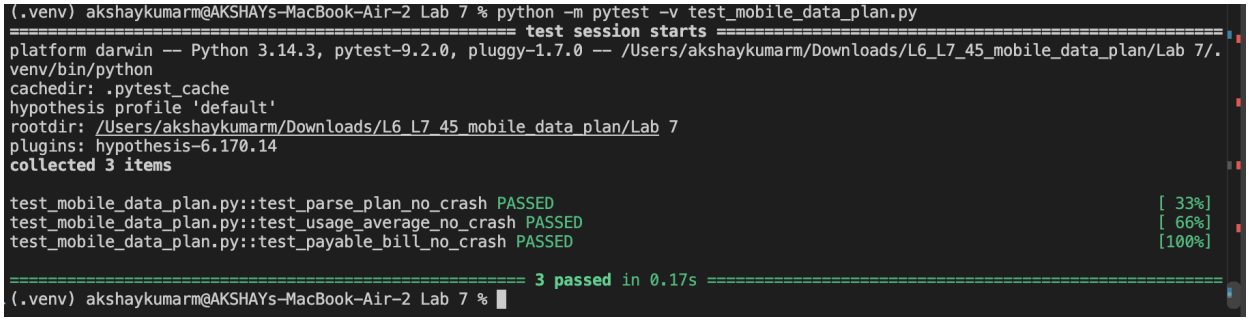


Figure 4. All three fuzz tests passing against the fixed code (3 passed in 0.17s).

All three tests passed. Hypothesis had also tried random text and other awkward strings during these runs, and none of them crashed the functions.

8. Summary of results

	Before the fix	**After the fix**
Normal input	Works	Works, same output
test_parse_plan_no_crash	Failed	Passed
test_usage_average_no_crash	Failed	Passed
test_payable_bill_no_crash	Failed	Passed
Total	**3 failed**	**3 passed**

9. What I learned

The bug was simple, but I would not have found it by testing by hand, because every input I could think of to type was a string. Hypothesis tried a value I never would have, and it showed the problem within a fraction of a second. The other useful part was the shrinking: instead of a long random string, the report said `s=None`, which pointed straight at the cause.

One limit is worth stating. My tests only check that the functions do not crash. They do not check that the answers are correct, so a function could return a wrong value and still pass. A stronger test suite would add properties about the results, for example that the average of a list always lies between its smallest and largest value.

Files submitted

1. This PDF with the screenshots
2. Fixed *mobile_data_plan.py*
3. Updated *test_mobile_data_plan.py*